

Bottom-Up Parsing

- Bottom-up parsing is a parsing method that starts with the input symbols (terminals) and builds the parse tree from the bottom (leaves) to the top (root).
 - It begins with the terminal symbols in the input.
 - These terminals are combined step by step into non-terminal symbols using the grammar rules.
 - This process continues until the start symbol is obtained.
 - In simple words, it recognizes the input string first and then reduces it into the start symbol.
 - Bottom-up parsing produces the rightmost derivation in reverse.

Working Process

- Read the input from left to right.
- Shift the input symbols one by one onto the stack.
- When the top of the stack matches the right-hand side (RHS) of a grammar rule, reduce it to the left-hand side (LHS) non-terminal.
- Repeat the shift and reduce steps until:
 - all input symbols are read, and
 - the stack contains only the start symbol.
- If the start symbol is obtained, the input string is accepted.

Common Bottom-Up Parsing Techniques

1. Shift-Reduce Parsing

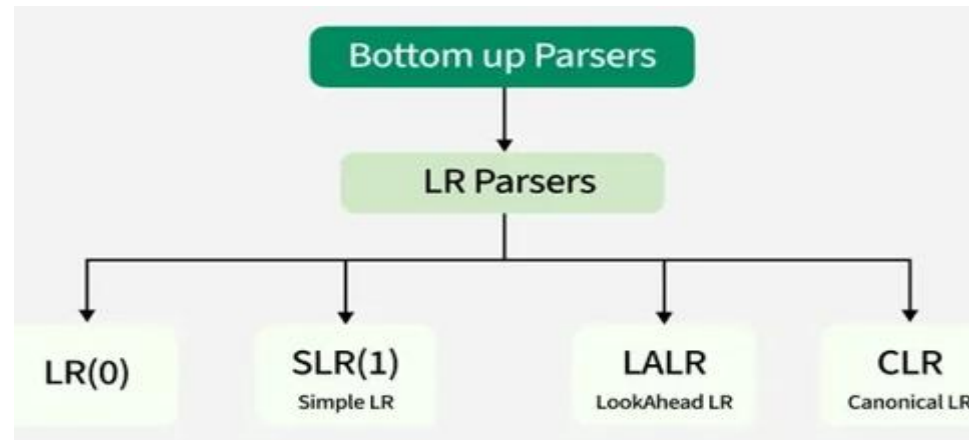
- Shift-reduce parsing is the basic bottom-up parsing method. It uses a stack and performs two main operations:
 - Shift: Move the next input symbol onto the stack.
 - Reduce: Replace a group of symbols (called a handle) on the stack with the corresponding non-terminal using a grammar rule.

2. Operator Precedence Parsing

- This method uses operator precedence (priority) to decide which operation should be performed first.
- It is mainly used for parsing arithmetic expressions involving operators like +, -, *, and /.

3. Table-Driven LR Parsing

- This method uses parsing tables called ACTION and GOTO to decide whether to shift, reduce, accept, or report an error.
- Types of LR Parsers:
 - ✓ LR(0)
 - ✓ SLR (Simple LR)
 - ✓ LR(1) / CLR(1) (Canonical LR)
 - ✓ LALR(1) (Look-Ahead LR)



Advantages

- Can handle a larger class of grammars than top-down parsers.
- Detects syntax errors as soon as possible.
- Efficient and widely used in compiler construction.
- Forms the basis for LR, SLR, LALR, and Canonical LR parsers.

Disadvantages

- More complex to implement than top-down parsing.
- Parser tables can become large for complex grammars.
- Manual construction of parsing tables is difficult.

Shift-Reduce Parser

- Shift-reduce parsing is a bottom-up parsing technique used in compiler design.
- It starts with the input tokens and builds the parse tree from the leaves to the root.
- The parser applies the grammar rules in reverse to reduce the input into the start symbol.
- It uses a stack to store symbols while parsing.

Actions Performed by a Shift-Reduce Parser

1. Shift

- Move the next input symbol onto the top of the stack.

2. Reduce

- Replace a group of symbols (handle) at the top of the stack with the corresponding non-terminal using a grammar rule.

3. Accept

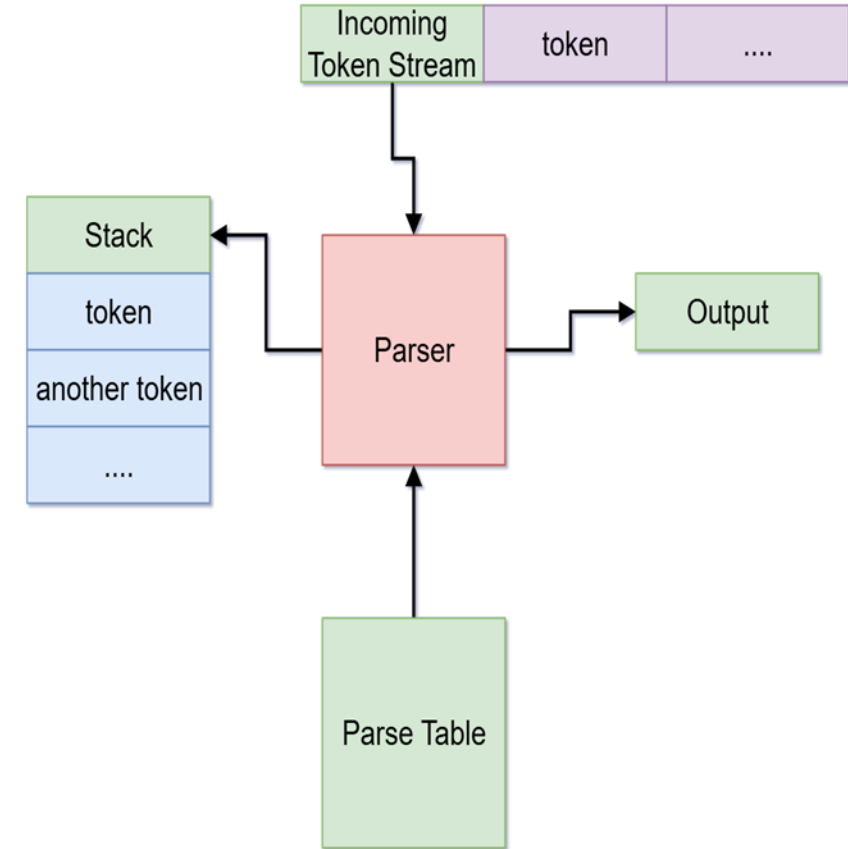
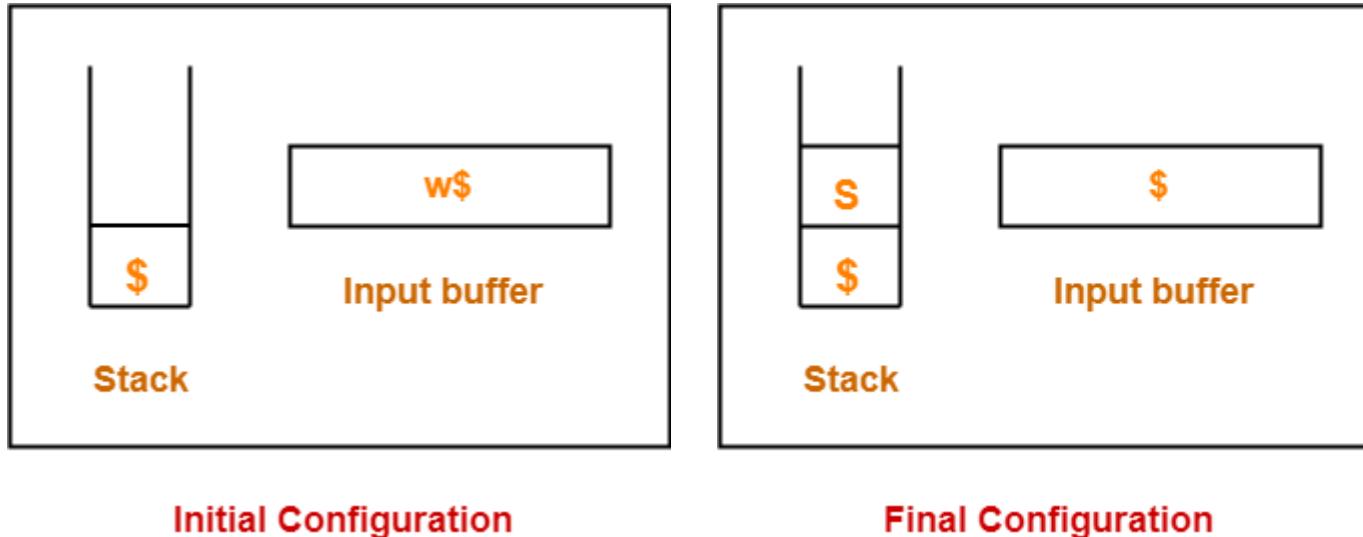
- The parser successfully completes parsing.
- This means the input string is valid according to the grammar.

4. Error

- The parser cannot decide what action to take. It cannot shift, reduce, or accept the input.
- This indicates that the input string is invalid or does not follow the grammar.

Shift-Reduce Parser: Data Structure

- A **Shift-Reduce Parser** uses two main data structures:
- **Stack**
 - Stores the grammar symbols during parsing.
 - Initially contains only the \$ (end marker).
- **Input Buffer**
 - Holds the input string to be parsed.
 - The input string is followed by the \$ symbol to indicate the end of input.



Working of a Shift-Reduce Parser

- Initially, the parser is in the following configuration:
 - Stack: \$
 - Input Buffer: input string \$
 - The parser performs the following operations repeatedly:
 - Shift Operation: Moves the next input symbol from the input buffer to the top of the stack.
 - Reduce Operation: When a handle (β) appears on the top of the stack, it is replaced by the corresponding non-terminal according to the grammar production.
 - The parser continues repeating the shift and reduce operations until one of the following conditions occurs:
 - Error: No valid shift or reduce operation is possible, so parsing fails.
 - Successful Parsing: The stack contains only the start symbol (S) and the \$ marker, while the input buffer is empty (contains only \$).
 - Final Configuration (Successful Parsing)
 - Stack: \$ S
 - Input Buffer: \$
- Note : What is handle in shift Reduce ?
A handle is a substring of the input that matches the right-hand side (RHS) of a production rule and can be reduced to the corresponding left-hand side (LHS) non-terminal.
- At this stage, the parser halts and reports that the input string has been successfully parsed.

1. Shift

- In the **shift** operation, the parser reads the **next input token** and pushes it onto the **stack**.
- The parser then moves **one position to the right** in the input stream.

Example:

Input: id + id

Stack before shift: \$

After shifting id: \$ id

The parser takes the first token id from the input, places it onto the stack, and moves the input pointer to the next token +.

2. Reduce

- In the **reduce** operation, the parser checks whether the symbols at the top of the stack match the **right-hand side (RHS)** of any grammar production.
- If a match is found, the parser replaces those symbols with the corresponding **left-hand side (LHS)** non-terminal.
- This process helps the parser gradually build larger grammar constructs until it reaches the start symbol.
- **Example:**
If the grammar contains:

$T \rightarrow id$

Then:

Stack before reduce: \$ id

Action: Reduce using $T \rightarrow id$

Stack after reduce: \$ T

3. Accept

The accept operation is performed when:

- ✓ The stack contains only the start symbol (along with the bottom-of-stack marker \$), and
- ✓ The input buffer is empty (except for the end-of-input marker \$).

This indicates that the input string has been successfully parsed and is syntactically correct according to the grammar.

Example:

Stack: \$ E (where E is the start symbol)

Input: \$

Action: Accept

Here

Since the parser has reduced the entire input to the start symbol and no input tokens remain, it accepts the input, confirming that the string is valid according to the given grammar.

4. Error

- An **error** occurs when the parser cannot perform any valid action, such as:
 - **Shift**
 - **Reduce**
 - **Accept**
- This happens when the current input token and stack contents do not match any valid grammar rule or parsing action.

When the parser reaches an invalid state where no shift or reduce operation is possible and the input cannot be accepted, it reports a **syntax error**.

This indicates that the input string does not follow the rules defined by the grammar.

Example 1: Shift-Reduce Parsing

• Grammar

$S \rightarrow S + S$

$S \rightarrow S - S$

$S \rightarrow (S)$

$S \rightarrow a$

Input String : $a - (a + a) \$$

Solution:

Shift-Reduce

Parsing Table

The given input string is parsed
Successfully.

Step	Stack	Input Buffer	Action
1	\$	$a - (a + a) \$$	Initial
2	\$ a	$- (a + a) \$$	Shift
3	\$ S	$- (a + a) \$$	Reduce $S \rightarrow a$
4	\$ S -	$(a + a) \$$	Shift
5	\$ S - (	$a + a) \$$	Shift
6	\$ S - (a	$+ a) \$$	Shift
7	\$ S - (S	$+ a) \$$	Reduce $S \rightarrow a$
8	\$ S - (S +	$a) \$$	Shift
9	\$ S - (S + a	$) \$$	Shift
10	\$ S - (S + S	$) \$$	Reduce $S \rightarrow a$
11	\$ S - (S	$) \$$	Reduce $S \rightarrow S + S$
12	\$ S - (S)	\$	Shift
13	\$ S - S	\$	Reduce $S \rightarrow (S)$
14	\$ S	\$	Reduce $S \rightarrow S - S$
15	\$ S	\$	Accept

Explanation

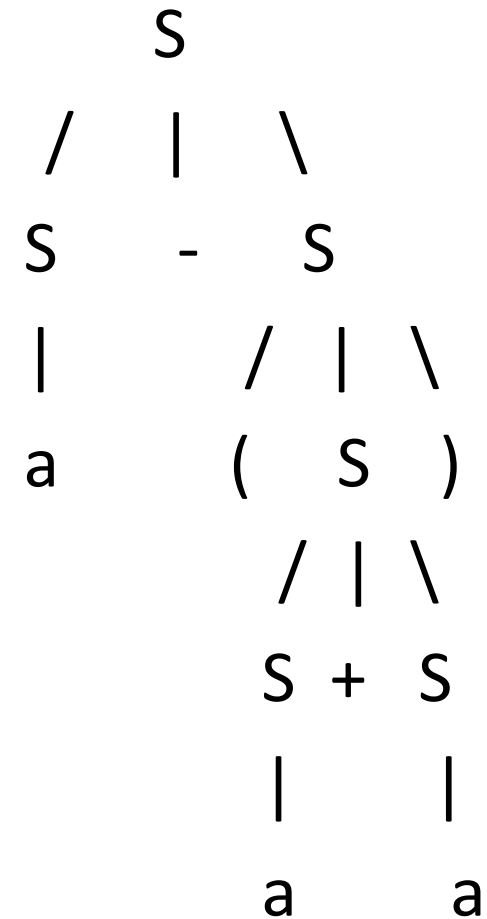
- Shift the first a and reduce it to S.
- Shift - and (.
- Shift the second a and reduce it to S.
- Shift +.
- Shift the third a and reduce it to S.
- Reduce $S + S$ to S.
- Reduce (S) to S.
- Reduce $S - S$ to S.
- The stack contains only the start symbol S and the input buffer is empty, so the parser accepts the input.
- Final Result: The input string $a_1 - (a_2 + a_3)$ is successfully parsed (Accepted).

Constructing Parse Tree

Input String : $a - (a + a) \$$

Step	Handle	Production Used
1	a	$S \rightarrow a$
2	a	$S \rightarrow a$
3	a	$S \rightarrow a$
4	$S + S$	$S \rightarrow S + S$
5	(S)	$S \rightarrow (S)$
6	$S - S$	$S \rightarrow S - S$
7	S	Accept

- Parse Tree



Example 2:

Given grammar

$$E \rightarrow E - E$$

$$E \rightarrow E \times E$$

$$E \rightarrow id$$

- Parse the input string:

$$id - id \times id$$

using a shift-reduce parser.

Solution:

Shift-Reduce Parsing Table

The given input string is parsed

Successfully.

Stack	Input Buffer	Action
\$	id - id × id \$	Initial
\$ id	- id × id \$	Shift
\$ E	- id × id \$	Reduce $E \rightarrow id$
\$ E -	id × id \$	Shift
\$ E - id	× id \$	Shift
\$ E - E	× id \$	Reduce $E \rightarrow id$
\$ E - E ×	id \$	Shift
\$ E - E × id	\$	Shift
\$ E - E × E	\$	Reduce $E \rightarrow id$
\$ E - E	\$	Reduce $E \rightarrow E \times E$
\$ E	\$	Reduce $E \rightarrow E - E$
\$ E	\$	Accept

Example 3:

Given Grammar

$E \rightarrow E + E$

$E \rightarrow E \times E$

$E \rightarrow id$

Input String :

$id * id + id \$$

Assumption:

Multiplication (*) has higher precedence than addition (+), so the expression is parsed as:

$(id * id) + id$

Solution:

Shift-Reduce Parsing Table

Stack	Input Buffer	Action
\$	id * id + id \$	Initial
\$ id	* id + id \$	Shift
\$ E	* id + id \$	Reduce $E \rightarrow id$
\$ E *	id + id \$	Shift
\$ E * id	+ id \$	Shift
\$ E * E	+ id \$	Reduce $E \rightarrow id$
\$ E	+ id \$	Reduce $E \rightarrow E * E$
\$ E +	id \$	Shift
\$ E + id	\$	Shift
\$ E + E	\$	Reduce $E \rightarrow id$
\$ E	\$	Reduce $E \rightarrow E + E$
\$ E	\$	Accept

The given input string is parsed Successfully.

Example 4:

Given Grammar

$E \rightarrow 2 E 2$

$E \rightarrow 3 E 3$

$E \rightarrow 4$

- Input String : 32423 \$

Solution:

Shift-Reduce Parsing Table

The given input string is parsed
Successfully.

Stack	Input Buffer	Action
\$	3 2 4 2 3 \$	Initial
\$ 3	2 4 2 3 \$	Shift
\$ 3 2	4 2 3 \$	Shift
\$ 3 2 4	2 3 \$	Shift
\$ 3 2 E	2 3 \$	Reduce $E \rightarrow 4$
\$ 3 2 E 2	3 \$	Shift
\$ 3 E	3 \$	Reduce $E \rightarrow 2E2$
\$ 3 E 3	\$	Shift
\$ E	\$	Reduce $E \rightarrow 3E3$
\$ E	\$	Accept

Example 4:

Given Grammar

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L , S \mid S$$

Input: $(a , (a , a)) \$$

Solution:

Shift-Reduce Parsing Table

Stack	Input Buffer	Action
\$	(a , (a , a)) \$	Initial
\$ (	a , (a , a)) \$	Shift
\$ (a	, (a , a)) \$	Shift
\$ (S	, (a , a)) \$	Reduce $S \rightarrow a$
\$ (L	, (a , a)) \$	Reduce $L \rightarrow S$
\$ (L ,	(a , a)) \$	Shift
\$ (L , (	a , a)) \$	Shift
\$ (L , (a	, a)) \$	Shift
\$ (L , (S	, a)) \$	Reduce $S \rightarrow a$
\$ (L , (L	, a)) \$	Reduce $L \rightarrow S$
\$ (L , (L ,	a)) \$	Shift
\$ (L , (L , a	)) \$	Shift
\$ (L , (L , S	)) \$	Reduce $S \rightarrow a$
\$ (L , (L	)) \$	Reduce $L \rightarrow L , S$
\$ (L , S)	) \$	Reduce $L \rightarrow S$
\$ (L , S)	\$	Shift
\$ S	\$	Reduce $S \rightarrow (L)$
\$ S	\$	Accept



Operator Precedence Parser

- Type of bottom-up parser useful for operator grammar.
- Parser : A parser that reads and understands operator precedence grammar is called as operator precedence parser.

Operator Precedence Parsing

- Operator precedence parsing is a type of shift-reduce (bottom-up) parsing technique.
- It is applied to a small class of grammars called operator grammars.
- An operator precedence parser takes an operator grammar and an input string as input and generates a parse tree if the string belongs to the language.
- In operator precedence parsing, shift and reduce operations are performed based on the precedence relationship between the top terminal symbol of the stack and the current input symbol.
- The parser uses an operator precedence table to decide whether to shift, reduce, or report an error.
- An operator precedence parser is a bottom-up parser that interprets operator grammars.

Operator Precedence Relations

- Three precedence relations are used:

Symbol	Meaning
<·	Yields precedence to (Shift)
=·	Equal precedence
·>	Takes precedence over (Reduce)

Key Features:

- It is a special type of **shift-reduce parser** that uses **bottom-up parsing technique**.
- It uses **precedence relations between terminals** to decide shift and reduce operations.
- It is mainly used for parsing **arithmetic expressions**.
- It requires an **operator grammar** and an **operator precedence table**.
- It resolves operator order using **precedence and associativity rules**.
- It is simple and efficient for arithmetic expression parsing.
- It cannot handle all types of grammars.

Limitations of Operator Precedence Parsing

- It works only for a limited class of grammars called **operator grammars**.
- It cannot handle grammars containing:
 - ϵ -productions
 - Consecutive non-terminals on the RHS
- It is not suitable for complex programming language constructs.

Operator Grammar

- An **operator grammar** is a type of context-free grammar used for designing operator precedence parsers.
- A grammar is considered an operator grammar if it satisfies the following conditions:
 - It should **not contain ϵ (empty string) productions**.
 - Production like $E \rightarrow \epsilon$ is Not allowed.
 - This means no production rule should generate an empty string.
 - No production rule should have **two consecutive non-terminals on the right-hand side (RHS)**.
 - Production such as $S \rightarrow AB$ is Not allowed
 - The RHS of a production should not contain adjacent variables (non-terminals).
- **Example :**

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow \text{id}$$

- In this grammar, there are no ϵ -productions, and no production contains two consecutive non-terminals.
- Therefore, it satisfies the conditions of an operator grammar.
- It can be used to define the precedence and associativity of operators during parsing.

Example 1: Operator Grammar

Given grammar:

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow \text{id}$$

This is an operator grammar because:

- It does not contain any ϵ -production.
- It does not contain two adjacent non-terminals on the RHS.

Example 2: Not an Operator Grammar

$$S \rightarrow S A S \mid a$$

$$A \rightarrow b S b \mid b$$

This is **not an operator grammar** because the production:

$S \rightarrow S A S$ contains adjacent non-terminals: $S A S$

Conversion into Operator Grammar

The grammar in Example2 can be converted into operator grammar by replacing the non-terminal A with its productions.

Given grammar:

$$S \rightarrow S A S \mid a$$

$$A \rightarrow b S b \mid b$$

Substituting A in $S \rightarrow S A S$:

$$S \rightarrow S b S b S \mid S b S \mid a$$

- Now, every pair of non-terminals is separated by a terminal symbol (b), so the grammar satisfies the conditions of an operator grammar.

Working of an Operator Precedence Parser

- An Operator Precedence Parser works by comparing the topmost terminal symbol on the stack with the current input symbol.
- Based on the precedence relationship between these two symbols, the parser decides the next action.

Possible Actions

1. Shift

- Push the current input symbol onto the stack when it has higher precedence.
- This is indicated by the precedence relation $<\cdot$ (yields precedence) or sometimes $=\cdot$ (equal precedence).

2. Reduce

- Replace a **handle** on the stack with its corresponding non-terminal.
- Performed when the stack symbol has higher precedence.
- This is indicated by the precedence relation $\cdot >$ (takes precedence).

3. Accept

- The parser accepts the input when the entire input string has been successfully parsed.
- At this stage, the stack contains the start symbol and \$ while input buffer contains only \$.

4. Error

- Report a syntax error when no valid precedence relation or parsing action exists.

Parsing Process

The parser repeatedly performs the following steps:

- Initialize the stack with the symbol \$.
- Place the input string in the input buffer, ending with \$.
- Compare the topmost terminal in the stack with the current input symbol.
- Use the operator precedence table to determine the appropriate action:
 - Shift if the stack terminal yields precedence ($<\cdot$) or has equal precedence ($=\cdot$) to the input symbol.
 - Reduce if the stack terminal takes precedence ($\cdot>$) over the input symbol.
 - Report an Error if no valid precedence relation exists.
- Repeat the process until one of the following conditions is reached:
 - Accept: The input string is completely parsed.
 - Error: A syntax error is detected.

Designing an Operator Precedence Parser

An operator precedence parser is designed using two main steps:

1. Define Precedence Relations

- Determine the precedence relationship between pairs of terminal symbols.
- The relations are:
 - $<$ (Yields Precedence): Left terminal has lower precedence $\rightarrow$ Shift.
 - $=$ (Equal Precedence): Both terminals have equal precedence.
 - $>$ (Takes Precedence): Left terminal has higher precedence $\rightarrow$ Reduce.

2. Construct Operator Precedence Table

- Arrange the precedence relations in a table format.
- The parser uses this table to decide whether to shift, reduce, accept, or report an error during parsing.

Parsing a Given String Using Operator Precedence Parser

Step 1: Insert End Markers

- Add \$ at the beginning and end of the input string.
- Use the operator precedence table to determine relations between symbols.

Example: $\text{id} + \text{id} \rightarrow \$ \text{id} + \text{id} \$$

Step 2: Find > Relation

- Scan the string from left to right until the first > relation is found.

Step 3: Find Matching < Relation

- Scan backward from > to find the nearest < relation.

Step 4: Identify and Reduce Handle

- The symbols between < and > form the handle.
- Replace the handle with the corresponding non-terminal.

Step 5: Repeat

- Continue the process until the start symbol is obtained.
- If the input is completely consumed, the string is accepted; otherwise, an error is reported.

Example 1: Operator Precedence Parsing

Given Grammar

$$E \rightarrow E A E \mid id$$

$$A \rightarrow + \mid x$$

- Input string: id + id x id

Add end markers:

$$id + id x id \$$$

Step 1:

Construct Operator Precedence Relations

The terminals in the grammar are:

$$id, +, x, \$$$

Precedence Rules

- id has the highest precedence.
- \$ has the lowest precedence.
- x (multiplication) has higher precedence than +.
- Also both operators are left associative.

So Operator Precedence Relations becomes:

$$\$ < id, +, x$$

$$id > +, x, \$$$

$$+ < id, x$$

$$+ > +, \$$$

$$x < id$$

$$x > +, x, \$$$

Operator precedence table :

Stack Top ↓ / Input→	id	+	x	\$
\$	<	<	<	
id		>	>	>
+	<	>	<	>
x	<	>	>	>

Step 2: Parse Input String

- Input: id + id x id \$
- Shift-Reduce Parsing Steps

Final Result

The stack contains only: \$ E
and the input is: \$

Therefore:

The string id + id x id is
successfully parsed
(Accepted).

Stack	Input	Relation	Action
\$	id + id x id \$	\$ < id	Shift id
\$ id	+ id x id \$	Id > +	Reduce id → E
\$ E	+ id x id \$	\$ < +	Shift +
\$ E +	id x id \$	+ < id	Shift id
\$ E + id	x id \$	id > x	Reduce id → E
\$ E + E	x id \$	+ < x	Shift x
\$ E + E x	id \$	x < id	Shift id
\$ E + E x id	\$	id > \$	Reduce id → E
\$ E + E x E	\$	x > \$	Reduce E x E → E
\$ E + E	\$	+ > \$	Reduce E + E → E
\$ E	\$	Accept condition	Accept

Explanation:

The root node is E because the entire input is reduced to the start symbol.

- The root production used is:

$$E \rightarrow E + E$$

- The right child E is further expanded using:

$$E \rightarrow E \times E$$

because multiplication (x) has higher precedence than addition (+).

- The leaf nodes are the terminal symbols:

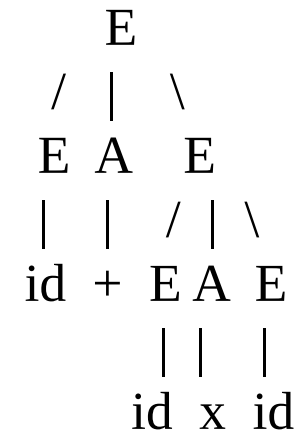
id, +, id, x, id

This parse tree represents the structure:

id + (id x id)

which matches the operator precedence rules.

Parse Tree



Example 2:

Given Grammar

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

Parse input : id + id * id \$

Solution:

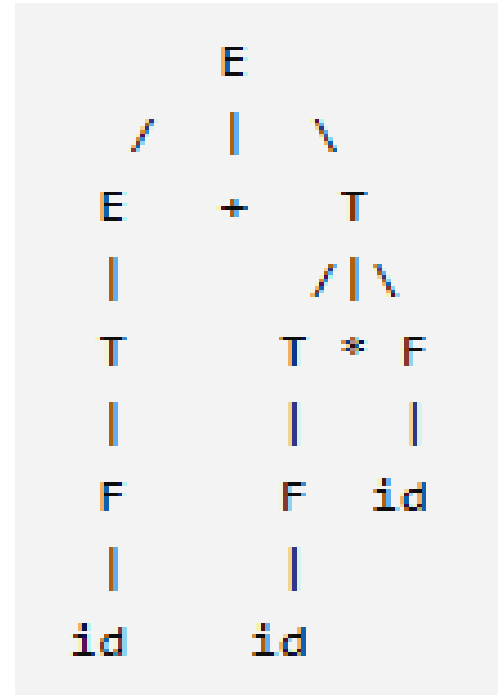
Step1 : Construct Operator precedence table

Stack / Input	id	+	*	(	)	\$
id		>	>		>	>
+	<	>	<	<	>	>
*	<	>	>	<	>	>
(	<	<	<	<	=	
)		>	>		>	>
\$	<	<	<	<		

Parsing Table: id + id * id \$

Stack	Input	Relation	Action
\$	id+id*id\$	\$ < id	Shift id
\$ id	+id*id\$	id > +	Reduce id → F → T → E
\$ E	+id*id\$	\$ < +	Shift +
\$ E +	id*id\$	+ < id	Shift id
\$ E + id	*id\$	id > *	Reduce id → F → T
\$ E + T	*id\$	+ < *	Shift *
\$ E + T *	id\$	* < id	Shift id
\$ E + T * id	\$	id > \$	Reduce id → F
\$ E + T * F	\$	* > \$	Reduce T * F → T
\$ E + T	\$	+ > \$	Reduce E + T → E
\$ E	\$	Accept	Parsing successful

Parse Tree



Using Another Method

Parsing $\text{id} + \text{id} * \text{id} \$$ by reducing handles

Solution:

- Insert End Markers : $\$ \text{id} + \text{id} * \text{id} \$$
- Insert precedence relations

$$\$ < \text{id} > + < \text{id} > * < \text{id} > \$$$

Now Find and Reduce Handles

- Reduce each $\text{id} \rightarrow E$

$$\Rightarrow \$ E + < \text{id} > * < \text{id} > \$$$

$$\Rightarrow \$ E + E * < \text{id} > \$$$

- Reduce second $\text{id} \rightarrow E$

$$\Rightarrow \$ E + E * E \$$$

- Reduce third $\text{id} \rightarrow E$
 $\Rightarrow \$ E + E \$$

- Reduce $E * E \rightarrow E$
 $\Rightarrow \$ E \$$

- Reduce $E + E \rightarrow E$
 $\Rightarrow \$ \$$
Accept

Example 3: Same Grammar as in Example 2

Parse input : (id+id)*id

Solution:

- Insert End Markers : \$ (id + id) * id \$

- Insert precedence relations

$$\$ < (< id > + < id >) < * < id > \$$$

Now Find and Reduce Handles

- Reduce first id

$$\$ < (< F > + < id >) < * < id > \$$$
$$\$ < (< T > + < id >) < * < id > \$$$
$$\$ < (< E > + < id >) < * < id > \$$$

- Reduce second id

$$\$ < (< E > + < F >) < * < id > \$$$
$$\$ < (< E > + < T >) < * < id > \$$$

- Reduce E + T

$$\$ < (< E >) < * < id > \$$$

- Reduce (E)

$$\$ < F < * < id > \$$$
$$\$ < T < * < id > \$$$

- Reduce last id

$$\$ < T < * < F > \$$$

- Reduce T * F

$$\$ < T > \$$$
$$\$ < E > \$$$

- Final

$$\$ E \$$$

Accept

Self Practices:

id - id / id

id*(id+id)

Example 4:

Given Grammar

$S \rightarrow (L) \mid a$

$L \rightarrow L , S \mid S$

Input string: (a , (a , a))

Solution:

Step 1: Construct Operator Precedence Relations as

	a	(	,	)	\$
a		>	>	>	>
(	<	>	>	>	>
)	<	>	>	>	>
,	<	<	>	>	>
\$	<	<	<	<	

Step 2: Insert End Markers

Add \$ at both ends: \$ (a , (a , a)) \$

Insert precedence relations:

$\$ < (< a > , < (< a > , < a >) >) > \$$

Step 2: Scan and Reduce Handles

- Reduce first a: $\$ < (S , < (< a > , < a >) >) > \$$

- Reduce a inside the nested list:

$\$ < (S , < (S , < a >) >) > \$$

- Reduce next a: $\$ < (S , < (S , S) >) > \$$

- Reduce S , S using: $L \rightarrow L, S$

$\$ < (S , < (L) >) > \$$

- Reduce (L) using: $S \rightarrow (L)$

$\$ < (S , S) > \$$

- Reduce outer S , S: $L \rightarrow L, S$

$\$ < (L) > \$$

- Reduce outer (L): $S \rightarrow (L)$

$\$ < S > \$$

- Final reduction:

$\$ S \$$

Step 3: Accept

Since the stack contains the start symbol S and the input is completely consumed:

$\$ S \$$

Input string is accepted.

Homework

Practice 1:

Grammar:

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow (E)$$

$$E \rightarrow \text{id}$$

Parse Following Inputs:

id + id * id \$

id + (id * id) \$

Practice 2:

Grammar

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow \text{id}$$

Input: id + id * id + id \$

General Rules for Operator Precedence Relations

1. Operand (id, a) Rule

- An operand is usually reduced before an operator or closing symbol.

Examples:

id > + id > * a > , a >)

Meaning: the parser has completed the operand and should reduce it.

2. Operator Before Operand Rule

- An operator or separator is usually shifted when followed by an operand.

Examples:

+ < id * < id , < a

Meaning: read the next operand before reducing.

3. Higher Precedence Operator Rule

- An operator with higher precedence takes precedence over a lower precedence operator.

Example:

For: * has higher precedence than +

Relations: * > + + < *

For: / has higher precedence than -

Relations: / > - - < /

4. Parentheses Rule

An opening parenthesis always shifts:

(< id (< + (< *

A closing parenthesis usually causes reduction:

id >) + >) * >)

Matching parentheses have equal precedence:

- (=)

5. Separator Rule (Comma, etc.)

For list-type grammars:

Before a separator → reduce the previous element:

 a > , id > ,

After a separator → shift the next element:

 , < a , < id

6. End Marker (\$) Rule

\$ has the lowest precedence.

Beginning: \$ < id \$ < (

Ending: id > \$) > \$

Completion:

 \$ = \$

Rules Summary

Symbol	Usually
Operand (id, a)	Highest precedence Reduce (>)
Arithmetic operator (+, -)	Lower precedence → Shift before higher-precedence operators
Higher-precedence operator (*, /)	Reduce before lower- precedence operators
, (separator)	Before , → Reduce (>), After , → Shift (<)
(	Shift (<)
)	Reduce (>)
(and)	Equal (=)
\$	Lowest precedence

LR Parser: Introduction

- LR parsing is a powerful bottom-up parsing technique widely used in compiler design.
- It is a type of shift-reduce parser that scans the input from left to right and constructs the rightmost derivation in reverse.
- The main purpose of an LR parser is to check whether the sequence of tokens produced by the lexical analyzer follows the grammar rules of a programming language.
- If the input is syntactically correct, the parser creates a parse tree, which is then used in later stages of the compiler.
- An LR parser makes parsing decisions by using a parsing table and a stack.
- The stack stores grammar symbols and parser states.
- The parsing table tells the parser whether to shift the next input symbol, reduce a group of symbols using a grammar production, accept the input, or report a syntax error.
- LR parsers are preferred because they can handle a large class of context-free grammars, detect syntax errors as soon as possible, and parse input efficiently in linear time.
- For these reasons, LR parsing techniques are widely used in compiler construction tools such as YACC and Bison.

Meaning of LR(k)

L – Left-to-Right Scanning

- The parser reads the input from **left to right**.
- Tokens are processed in the order they appear in the source program.

• R – Rightmost Derivation

- The parser constructs the **rightmost derivation in reverse**.
- It starts with the input string and repeatedly reduces it until the **start symbol** is obtained.

• k – Look-ahead

- **k** represents the number of upcoming input tokens the parser examines before making a parsing decision.
- A larger value of **k** gives the parser more information but also makes the parsing table more complex.
- In practice, most LR parsers use **k = 1**, which means they look at **one input symbol ahead** before deciding the next parsing action.

Example

LR(1) means that the parser:

- Scans the input from left to right.
- Constructs the rightmost derivation in reverse.
- Uses one look-ahead token to decide the next parsing action.

Types of LR Parsers

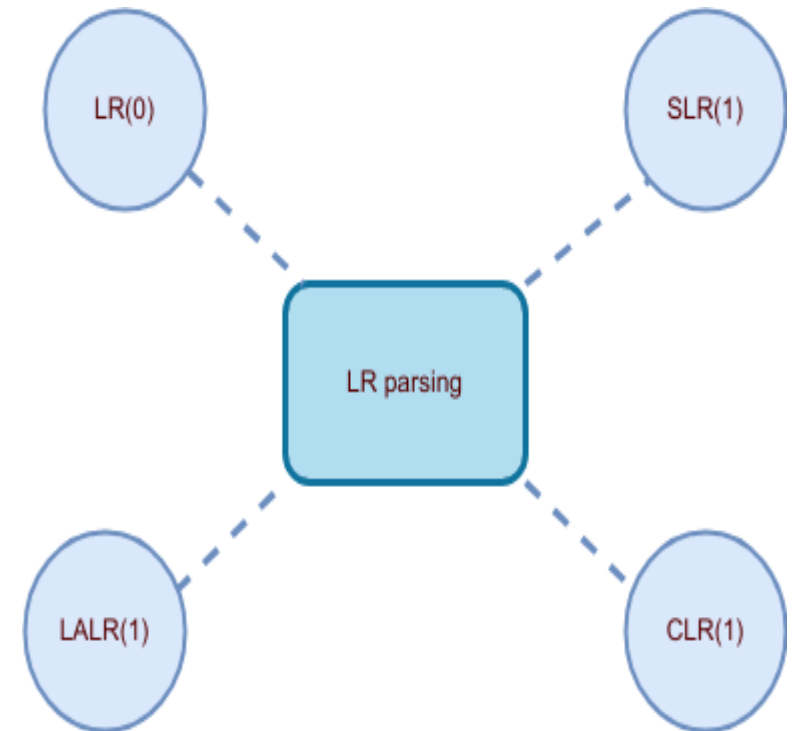
- LR parsers are classified into different types based on their parsing power, method of constructing parsing tables, and use of look-ahead symbols.
- The four main types of LR parsers are:
 - LR(0)
 - SLR(1) (Simple LR Parser)
 - CLR(1) (Canonical LR Parser)
 - LALR(1) (Look-Ahead LR Parser)

Order of LR Parser Power

They are arranged from simpler to more powerful:

LR(0) → SLR(1) → LALR(1) → CLR(1)

Less powerful → More powerful



As we move from LR(0) to CLR(1), the parser can recognize a larger number of grammars, but the complexity and size of the parsing table also increase.

Types of LR Parsers

LR(0) Parser

- It is the simplest type of LR parser.
- It does not use any look-ahead symbols while making parsing decisions.
- It has limited parsing power and can handle only a small class of grammars.

SLR(1) Parser (Simple LR Parser)

- SLR(1) is an improved version of LR(0).
- It uses one look-ahead symbol and FOLLOW sets of non-terminals to reduce conflicts.
- It can parse more grammars than LR(0) but is still less powerful than other LR parsers.

CLR(1) Parser (Canonical LR Parser)

- CLR(1) is the most powerful standard LR parsing technique.
- It uses one look-ahead symbol for each item while constructing the parsing table.
- It can handle a larger class of grammars but requires more memory because of larger parsing tables.

LALR(1) Parser (Look-Ahead LR Parser)

- LALR(1) is a modified version of CLR(1).
- It combines similar states of CLR(1) to reduce the size of the parsing table.
- It provides almost the same parsing power as CLR(1) with smaller tables.
- It is widely used in practical compiler tools such as YACC.

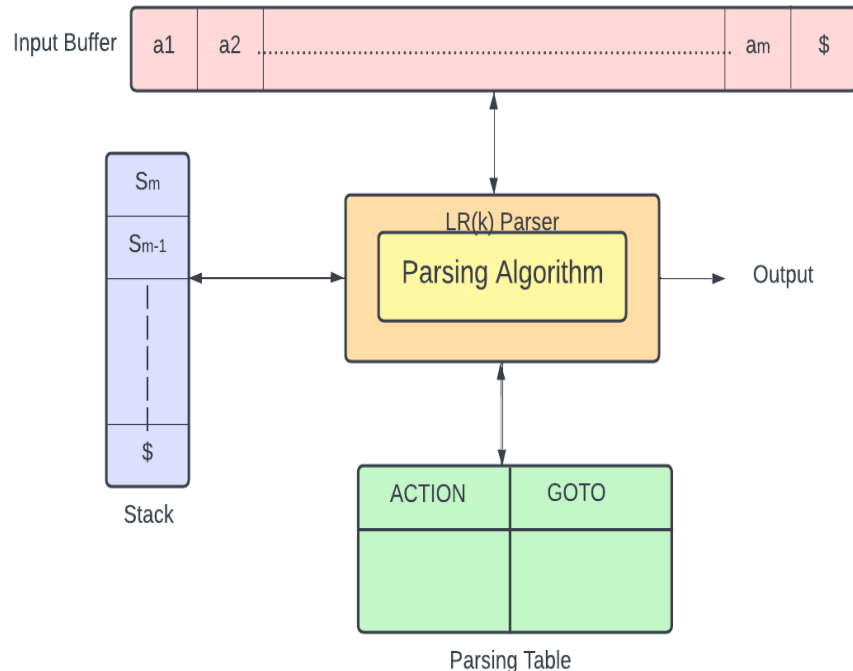
Structure of an LR Parser

- An **LR parser** is a **table-driven bottom-up parser** used in compiler design to analyze the syntax of a program.
- It reads the input string from **left to right** and constructs the **rightmost derivation in reverse order**.
- The main purpose of an LR parser is to determine whether the given input string belongs to a particular grammar.
- It performs this task using three main elements: a **stack**, an **input buffer**, and **parsing tables**.
- The parser makes decisions based on the **current state stored in the stack** and the **next input symbol**
- According to the parsing table, it performs one of four actions:
 - **Shift:** Moves the next input symbol onto the stack and changes the parser state.
 - **Reduce:** Replaces a sequence of symbols on the stack with the corresponding grammar rule.
 - **Accept:** Indicates that the input string is valid and follows the grammar.
 - **Error:** Indicates that the input string contains a syntax error.

Structure of an LR Parser

LR parser consists of the following components:

- Input Buffer
- Stack
- Parsing Table (ACTION and GOTO)
- Parser Driver (Control Program)
- Output



Components of an LR Parser

1. Input Buffer

- The input buffer stores the sequence of tokens generated by the lexical analyzer.
- The parser reads tokens from the buffer one at a time from left to right.
- The end of the input is marked with a special symbol, usually $\$$.

2. Stack

- The stack stores grammar symbols and parser states during parsing.
- It keeps track of the current progress of the parsing process.
- The parser uses the stack to perform shift and reduce operations.

3. Parsing Table

- The parsing table contains the instructions needed by the parser.
- It is divided into two parts:

ACTION Table

- Determines the action to be performed for a given state and input symbol.
- It specifies whether the parser should shift, reduce, accept, or report an error.

GOTO Table

- Determines the next state after a reduction.
- It is used when a non-terminal symbol is placed on the stack.

4. Parser Driver (Control Program)

- The parser driver controls the overall parsing process.
- It reads information from the stack, input buffer, and parsing tables.
- It performs the required action until the input is accepted or an error is detected.

5. Output

- The output of the LR parser is usually a **parse tree** or a sequence of reductions.
- This output is used by later compiler phases such as semantic analysis and code generation.

Working of an LR Parser

- The LR parsing algorithm works by reading the input string from left to right and constructing the rightmost derivation in reverse order.
- The parser repeatedly checks the current state and the next input symbol and uses the ACTION and GOTO tables to decide the next step.
- The parsing process continues until the entire input is successfully parsed or a syntax error is detected.
- It follows the shift-reduce parsing technique.

Shift-Reduce Parsing

- In shift-reduce parsing, the parser uses a stack to store grammar symbols and states while processing the input.
- At each step, the parser performs one of the following operations:

1.Shift

- The parser reads the next input symbol and pushes it onto the stack.
- A new state is also added to the stack.
- The input pointer moves to the next symbol.

2. Reduce

- The parser replaces a sequence of symbols on the stack that matches the right side of a grammar production with the corresponding non-terminal on the left side.

3. Accept

- The parser accepts the input when the complete string has been successfully reduced to the start symbol.
- It indicates that the input string is valid according to the grammar.

4. Error

- If no valid action is found in the parsing table for the current state and input symbol, the parser reports a syntax error.

LR Parser Algorithm

The LR parsing algorithm is the same for all LR parsers, but the parsing table (ACTION and GOTO) differs depending on the parser type.

- Input Buffer
 - Stores the input string to be parsed.
 - The input ends with a special end marker \$.
- Stack
 - Stores parser states and grammar symbols.
 - The current stack state and the next input symbol are used to determine the parser's action.
- Parsing Table
 - A two-dimensional table divided into ACTION and GOTO parts.
 - ACTION Table:
 - Specifies whether to shift, reduce, accept, or report an error based on the current state and input terminal.
 - GOTO Table:
 - Indicates the next state after a reduction when a non-terminal is placed on the stack.

LR Parsing Steps

Step 1: Augment the Grammar

- Add a new start production, such as $S' \rightarrow S$, to simplify the parsing process.

Step 2: Construct LR Items

- Create the **canonical collection of LR(0) or LR(1) items** using **closure** and **goto** operations.

Step 3: Construct DFA

- Build a **Deterministic Finite Automaton (DFA)** to represent transitions between LR item sets.

Step 4: Create Parsing Table

- Construct the **ACTION** and **GOTO** tables from the DFA.
- These tables guide the parser to perform **shift, reduce, accept, or error** actions.

Construction of LR(0) Parsing Table

- The **LR(0) parsing table** is constructed using the **canonical collection of LR(0) items** and the **DFA**.
- It consists of two parts: **ACTION** and **GOTO**.

Steps to Construct LR(0) Parsing Table

1. Augment the Grammar

- Add a new start production:
 $S' \rightarrow S$
- This helps the parser identify the completion of parsing.

2. Construct the Canonical Collection

- Generate all **LR(0) item sets** using the **closure** and **goto** operations.

3. Construct the DFA

- Create a **DFA** by connecting LR(0) item sets through transitions on grammar symbols.

Construction of LR(0) Parsing Table

- The **LR(0) parsing table** is constructed using the **canonical collection of LR(0) items** and the **DFA**.
- It consists of two parts: **ACTION** and **GOTO**.

Steps to Construct LR(0) Parsing Table

1. Augment the Grammar

- Add a new start production:
 $S' \rightarrow S$
- This helps the parser identify the completion of parsing.

2. Construct the Canonical Collection

- Generate all **LR(0) item sets** using the **closure** and **goto** operations.

3. Construct the DFA

- Create a **DFA** by connecting LR(0) item sets through transitions on grammar symbols.

4. Fill the ACTION Table

- If there is a transition on a **terminal symbol**, enter **Shift (S)** with the corresponding state.
- If an item has the form $A \rightarrow \alpha .$, enter **Reduce (R)** using that production.
- If the item $S' \rightarrow S .$ is present, enter **Accept** in the **\$ column**.
- If no valid action exists, enter **Error**.

5. Fill the GOTO Table

- For transitions on **non-terminal symbols**, enter the corresponding next state in the GOTO table.

6. Complete the Parsing Table

- The completed **ACTION** and **GOTO** tables together form the LR(0) parsing table.
- The parser uses this table to perform **shift, reduce, accept, and error** operations.

Augmented Grammar

- An **augmented grammar** is created by adding a new start production to the original grammar.
- For a grammar **G**, the augmented grammar **G'** is formed by introducing a new start symbol **S'** and adding the production:

$$S' \rightarrow S$$

where **S** is the original start symbol.

- The purpose of augmentation is to help the LR parser identify when the parsing process is complete.
- When the parser reaches the item **S' → S ·**, it means the entire input has been successfully parsed, and the parser performs the **Accept** action.

Example

Given Grammar (G):

$$S \rightarrow AA$$

$$A \rightarrow aA \mid b$$

Augmented Grammar (G'):

$$S' \rightarrow S$$

$$S \rightarrow AA$$

$$A \rightarrow aA \mid b$$

Here, **S'** is the new start symbol, and **S' → S** is called the augmented production.

The item **S' → S ·** indicates that parsing is complete.

Closure Function in LR(0) Parsing

- The Closure function is used to generate all possible LR(0) items for a given item set.
- It ensures that whenever the dot (.) appears before a non-terminal, all of that non-terminal's productions are included in the item set.

Definition

- If an item contains a non-terminal immediately after the dot, then add all productions of that non-terminal with the dot placed at the beginning.
- Repeat this process until no new items can be added.

Algorithm for Closure(I)

1. Start with the given set of LR(0) items I.
2. For every item of the form $A \rightarrow \alpha . B \beta$, where B is a non-terminal:
 - ✓ Add all items $B \rightarrow . \gamma$ for every production $B \rightarrow \gamma$.
3. Repeat Step 2 until no new items can be added.
4. The resulting set is called Closure(I).

Example1:

Grammar:

$$S' \rightarrow S$$

$$S \rightarrow AA$$

$$A \rightarrow aA \mid b$$

Find Closure($\{S' \rightarrow .S\}$)

Step 1:

$$S' \rightarrow .S$$

Since the dot is before S,
add all productions of S:

$$S' \rightarrow .S$$

$$S \rightarrow .AA$$

Now the dot is before A, so add all productions of A:

$$S' \rightarrow .S$$

$$S \rightarrow .AA$$

$$A \rightarrow .aA$$

$$A \rightarrow .b$$

The dot is now before terminals (a and b), so no more items are added.

Therefore,

Closure($\{S' \rightarrow .S\}$) =

$$S' \rightarrow .S$$

$$S \rightarrow .AA$$

$$A \rightarrow .aA$$

$$A \rightarrow .b$$

GOTO Operation in LR(0) Parsing

- The GOTO operation is used to find the next set of LR(0) items after reading a grammar symbol (terminal or non-terminal).
- It shows the **transition from one state to another** in the LR parser's DFA.

Definition

For an item set I and a grammar symbol X :

If there is an item of the form:

$$A \rightarrow \alpha \cdot X \beta \text{ in } I$$

then **GOTO(I, X)** is obtained by:

- Moving the dot (.) past X to get $A \rightarrow \alpha X \cdot \beta$.
- Applying the **Closure** function to the new set of items.

Algorithm for GOTO(I, X)

- Start with the given item set I and grammar symbol X .
- Find all items in which the dot (.) is immediately before X .
- Move the dot past X in each of those items.
- Apply the Closure function to the resulting set of items.
- The final set is called GOTO(I, X).

Example1:

Given grammar:

$$S' \rightarrow S$$
$$S \rightarrow AA$$
$$A \rightarrow aA \mid b$$

Suppose we have an item set:

I0:

$$S' \rightarrow \cdot S$$
$$S \rightarrow \cdot AA$$
$$A \rightarrow \cdot aA$$
$$A \rightarrow \cdot b$$

Find $GOTO(I0, S)$:

The item:

$S' \rightarrow \cdot S$ has S after the dot.

Move the dot past S:

$$S' \rightarrow S \cdot$$

Now apply closure (no new items are added).

Therefore:

$$GOTO(I0, S) = \{ S' \rightarrow S \cdot \}$$

Points to Remember

- ✓ Closure adds new items when the dot is before a non-terminal.
- ✓ GOTO moves the dot over a symbol and then applies closure.
- ✓ Together, Closure and GOTO are used to construct the canonical collection of LR(0) items and the DFA for LR parsing.

Canonical Collection of LR(0) Items

- An LR(0) item is a grammar production with a dot (.) placed at some position on the right-hand side (RHS) of the production.
- The dot (.) shows how much of the production has been processed by the parser.
- Each LR(0) item set represents a state in the LR(0) parser.
- The canonical collection of LR(0) items is the complete set of all LR(0) item sets generated by repeatedly applying the Closure and GOTO operations.
- These item sets are used to construct the DFA and the ACTION and GOTO parsing tables.
- In LR(0) parsing, a reduce action is entered for the entire row of the parsing table because LR(0) does not use lookahead symbols to decide when to reduce.

Example of LR(0) Item

Given production:

$$A \rightarrow aA$$

Possible LR(0) items are:

$A \rightarrow \cdot aA$ (Nothing has been read)
 $A \rightarrow a \cdot A$ (Symbol 'a' has been read)
 $A \rightarrow aA \cdot$ (Complete production, ready for reduction)

When the dot reaches the end of a production ($A \rightarrow \alpha \cdot$), the parser performs a reduce operation using that production.

Constructing LR(0) Items

Example 1:

Given Grammar:

$S \rightarrow AA$

$A \rightarrow aA \mid b$

Step 1: Augment the Grammar

Add a new start production:

$S' \rightarrow S$

$S \rightarrow AA$

$A \rightarrow aA \mid b$

Step 2:

Construct Canonical Collection of LR(0) Items

$I_0 = \text{Closure}(\{S' \rightarrow \cdot S\})$

Start with:

$S' \rightarrow \cdot S$

Since the dot is before S, add the production of S:

$S \rightarrow \cdot AA$

Since the dot is before A, add the productions of A:

$A \rightarrow \cdot aA$

$A \rightarrow \cdot b$

Therefore,

I_0 :

$S' \rightarrow \cdot S$

$S \rightarrow \cdot AA$

$A \rightarrow \cdot aA$

$A \rightarrow \cdot b$

$I_1 = \text{GOTO}(I_0, S)$

Move the dot past S:

$S' \rightarrow S \cdot$

So,

I_1 :

$S' \rightarrow S \cdot$

This indicates the accept state.

$I_2 = \text{GOTO}(I_0, A)$

From:

$S \rightarrow \cdot AA$

Move dot past the first A:

$S \rightarrow A \cdot A$

Since dot is before A, add A productions:

$A \rightarrow \cdot aA$

$A \rightarrow \cdot b$

Therefore,

I_2 :

$S \rightarrow A \cdot A$

$A \rightarrow \cdot aA$

$A \rightarrow \cdot b$

$I_3 = \text{GOTO}(I_0, a)$

From:

$A \rightarrow \cdot aA$

Move dot past a:

$A \rightarrow a \cdot A$

Since dot is before A, add A productions:

$A \rightarrow \cdot aA$

$A \rightarrow \cdot b$

Therefore,

I_3 :

$A \rightarrow a \cdot A$

$A \rightarrow \cdot aA$

$A \rightarrow \cdot b$

• $I_4 = \text{GOTO}(I_0, b)$

From: $A \rightarrow \cdot b$

Move dot past b: $A \rightarrow b \cdot$

This is a reduce item.

I_4 :

$A \rightarrow b \cdot$

$I_5 = \text{GOTO}(I_2, A)$

From:

$S \rightarrow A \cdot A$

Move dot past A: $S \rightarrow AA \cdot$

This is a reduce item.

I_5 :

$S \rightarrow AA \cdot$

$I_6 = \text{GOTO}(I_3, A)$

From: $A \rightarrow a \cdot A$

Move dot past A: $A \rightarrow aA \cdot$

This is a reduce item.

I_6 :

$A \rightarrow aA \cdot$

Final Canonical Collection of LR(0) Items

I_0 :

$S' \rightarrow \cdot S$

$S \rightarrow \cdot AA$

$A \rightarrow \cdot aA$

$A \rightarrow \cdot b$

I_1 :

$S' \rightarrow S \cdot$

I_2 :

$S \rightarrow A \cdot A$

$A \rightarrow \cdot aA$

$A \rightarrow \cdot b$

I_3 :

$A \rightarrow a \cdot A$

$A \rightarrow \cdot aA$

$A \rightarrow \cdot b$

I_4 :

$A \rightarrow b \cdot$

I_5 :

$S \rightarrow AA \cdot$

I_6 :

$A \rightarrow aA \cdot$

Now These item sets are used to draw the **DFA** and construct the **LR(0) parsing table**.

Drawing DFA:

The DFA contains the 7 states I_0 to I_6 .

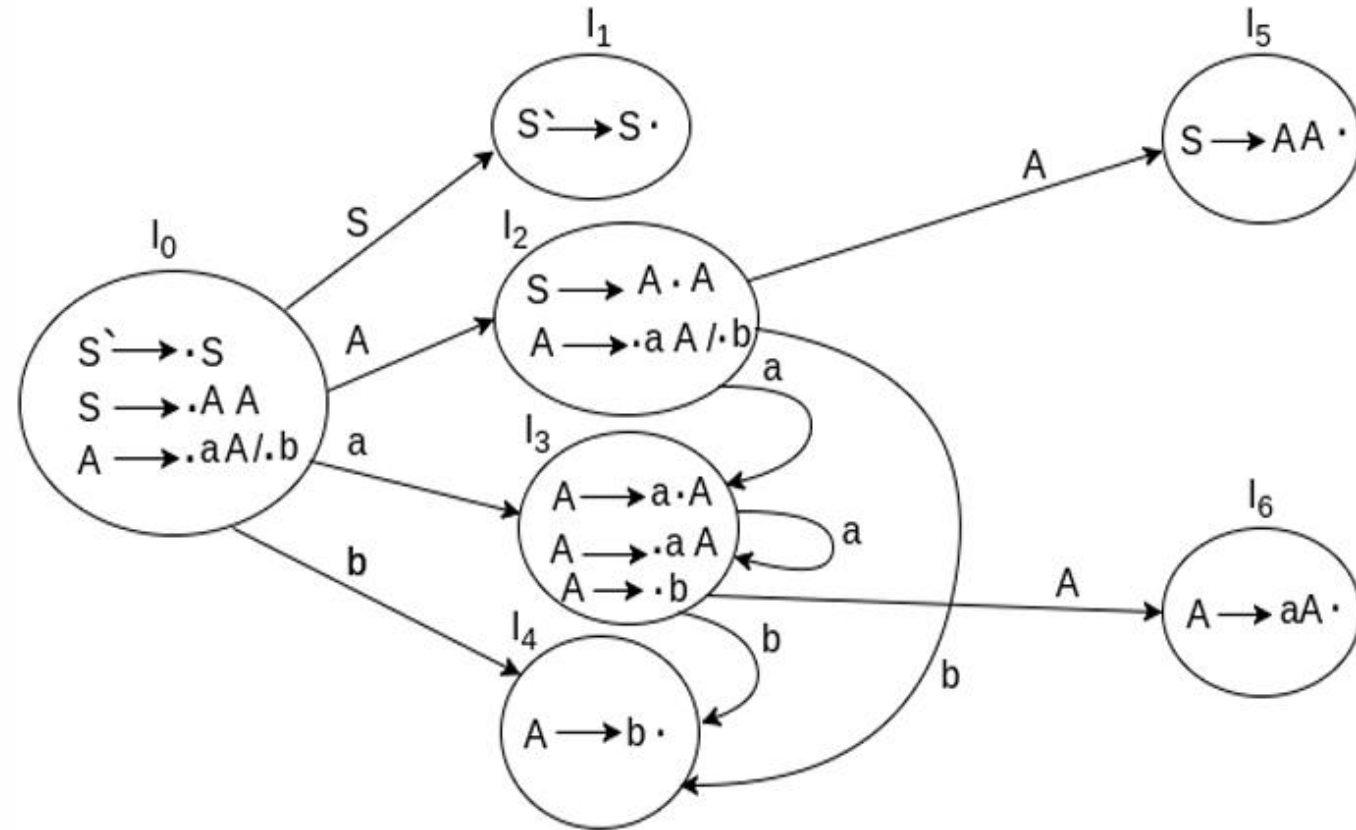


Fig: Resulting DFA

Current State	Input Symbol	Next State
I_0	S	I_1
I_0	A	I_2
I_0	a	I_3
I_0	b	I_4
I_2	A	I_5
I_2	a	I_3
I_2	b	I_4
I_3	A	I_6
I_3	a	I_3
I_3	b	I_4

Fig: Transition Table

HOW TO DRAW DFA??

- Start with I_0 .
- For **every symbol after the dot**, apply **GOTO**.
- Every new GOTO result becomes a **new state** ($I_1, I_2, \dots$).
- Draw an arrow from the current state to the new state and label it with the symbol used in GOTO.
- Repeat until no new states are generated.
- This is exactly the DFA used to construct the **LR(0) parsing table**.

LR(0) Parsing Table

The LR(0) parsing table is constructed from the DFA of LR(0) items.

It consists of two parts:

- **ACTION Table:** Contains Shift (S), Reduce (R), Accept, or Error actions for terminal symbols.
- **GOTO Table:** Contains the next state for non-terminal symbols after a reduction.

Rules for Constructing the LR(0) Parsing Table

• Shift Entry

- If a state has a transition to another state on a terminal, enter Shift (S) followed by the destination state in the ACTION table.
- Example: If $I_0 \xrightarrow{a} I_3$, then write S3 in the ACTION table for state I_0 and terminal a.

• GOTO Entry

- If a state has a transition to another state on a non-terminal, enter the destination state in the GOTO table.
- Example: If $I_0 \xrightarrow{A} I_2$, then write 2 in the GOTO table for state I_0 and non-terminal A.

• Reduce Entry

- If a state contains a completed item (the dot is at the end of a production), such as: $A \rightarrow \alpha$.
- Enter Reduce (R) by that production in all terminal columns, including \$.
- This is because LR(0) parsers do not use lookahead symbols.

• Accept Entry

- If a state contains the item: $S' \rightarrow S \cdot$
 - Enter Accept in the ACTION table under the \$ column.

State	ACTION			GOTO	
	a	b	\$	S	A
0	S3	S4	–	1	2
1	–	–	Accept	–	–
2	S3	S4	–	–	5
3	S3	S4	–	–	6
4	R3	R3	R3	–	–
5	R1	R1	R1	–	–
6	R2	R2	R2	–	–

Resulting LR(0) Parsing Table

Explanation of Table Entries

- S3 → Shift and go to State 3
- S4 → Shift and go to State 4
- R1 → Reduce by Production 1: $S \rightarrow AA$
- R2 → Reduce by Production 2: $A \rightarrow aA$
- R3 → Reduce by Production 3: $A \rightarrow b$
- Accept → Input string is successfully parsed.
- – → Error (no valid action)

Rule Summary

- Terminal transition → Shift (S)
- Non-terminal transition → GOTO
- Completed item ($\cdot$ at the end) → Reduce (R) in all terminal columns
- $S' \rightarrow S \cdot \rightarrow$ Accept under \$

Parsing input using LR(0) Parse Table

For given grammar:

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

And input aabb\$

Reduction numbers:

R1: $S \rightarrow AA$

R2: $A \rightarrow aA$

R3: $A \rightarrow b$

Parsing Table:

Steps	Stack	Input	Action
1	0	aabb\$	S3
2	0 a 3	abb\$	S3
3	0 a 3 a 3	bb\$	S4
4	0 a 3 a 3 b 4	b\$	R3 : $A \rightarrow b$
5	0 a 3 a 3 A 6	b\$	R2 : $A \rightarrow aA$
6	0 a 3 A 6	b\$	R2 : $A \rightarrow aA$
7	0 A 2	b\$	S4
8	0 A 2 b 4	\$	R3 : $A \rightarrow b$
9	0 A 2 A 5	\$	R1 : $S \rightarrow AA$
10	0 S 1	\$	Accept

Parsing Explanation:

Step 1:

Current state = 0.
Read **a**.
Apply **S3**.
Push **a 3** onto the stack.

Step 2:

Stack ends with **a 3**.
Read **a**.
Apply **S3**.
Push **a 3** onto the stack.

Step 3:

Stack ends with **a 3**.
Read **b**.
Apply **S4**.
Push **b 4** onto the stack.

Step 4:

Stack ends with **b 4**.
Apply **R3: A → b**.
Pop **b 4**, push **A**,
then **GOTO(3, A) = 6**.

Step 5:

Stack ends with **A 6**.
Apply **R2: A → aA**.
Pop **a 3 A 6**, push **A**, then **GOTO(3, A) = 6**.

Step 6:

Stack ends with **A 6**.
Apply **R2: A → aA**.
Pop **a 3 A 6**, push **A**, then **GOTO(0, A) = 2**.

Step 7:

Stack ends with **A 2**.
Read **b**.
Apply **S4**.
Push **b 4** onto the stack.

Step 8:

Stack ends with **b 4**.
Apply **R3: A → b**.
Pop **b 4**, push **A**, then **GOTO(2, A) = 5**.

Step 9:

Stack ends with **A 5**.
Apply **R1: S → AA**.
Pop **A 2 A 5**, push **S**, then **GOTO(0, S) = 1**.

Step 10:

Stack ends with **S 1**.
Read **\$**.
Apply **Accept**.
The input string **aabb** is
accepted

Homework :LR(0) Parsing

Practice 1:

Given Grammar:

$$S \rightarrow AA$$

$$A \rightarrow aA$$

$$A \rightarrow b$$

Parse following inputs:

$$w1 = abbb$$

$$w2 = aab$$

$$W3 = aaa$$

Practice 2:

Given Grammar:

$$S \rightarrow AB$$

$$A \rightarrow aA \mid b$$

$$B \rightarrow c$$

Parse following inputs:

$$w1 = abc$$

$$W2 = aaabc$$